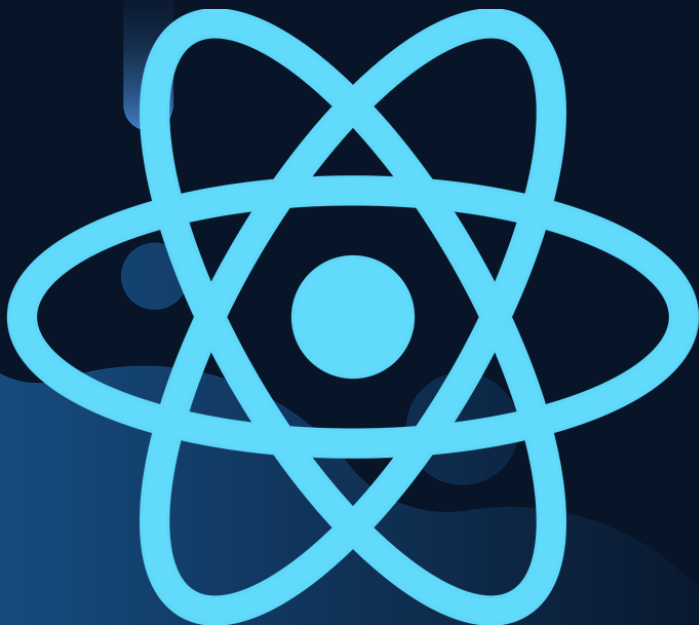


REACT LIFECYCLE METHODS FOR **FUNCTION BASE COMPONENTS**

VS

CLASS BASED COMPONENTS



Class Based Components

Lifecycle methods

Class-based components have lifecycle methods that allow you to perform actions at various points in a component's life, like initialization, updates, and unmounting. Some of the essential lifecycle methods are:

- **constructor():** It is the first method called when a component is created. It's used for initializing state and binding event handlers.
- **componentDidMount():** This method is invoked once the component has been rendered on the screen. It is often used for side effects like data fetching from APIs.
- **componentDidUpdate():** This method is called when the component's state or props change, after a re-render. It is useful for performing additional actions based on the changes.
- **componentWillUnmount():** This method is called just before the component is removed from the DOM. It's typically used to clean up resources, event listeners, etc.

Function Based Components

Lifecycle methods



React introduced hooks as a way to use state and lifecycle features in function-based components without using classes. The most commonly used hooks for lifecycle behavior are **useEffect** and **useState**.

- **useState:** It allows you to add state to a functional component by returning the state value and a function to update it.
- **useEffect:** This hook replaces **componentDidMount**, **componentDidUpdate**, and **componentWillUnmount**. It allows you to perform side effects and manage subscriptions inside a functional component.



Constructor vs useEffect



Constructor

```
1  constructor(props) {  
2    super(props);  
3    this.state = {  
4      count: 0,  
5    };  
6  }
```



useEffect

```
1  const [count, setCount] = useState(0);
```

ComponentDidMount, ComponentWillUnmount vs useEffect



ComponentDidMount and ComponentWillUnmount

```
1  componentDidMount() {  
2    console.log('Component has mounted.');
```



```
3  }  
4  
5  componentWillUnmount() {  
6    console.log('Component is about to be unmounted.');
```



```
7  }
```



useEffect

```
1  useEffect(() => {  
2    console.log('Component has mounted.');
```



```
3    return () => {  
4      console.log('Component is about to be unmounted.');
```



```
5    };  
6  }, []);
```

ComponentDidUpdate vs useEffect



componentDidUpdate

```
1  componentDidUpdate() {  
2    console.log('Component has updated.');
```

```
3  }
```



useEffect

```
1  useEffect(() => {  
2    console.log('Component has updated.');
```

```
3  }, [count]);
```

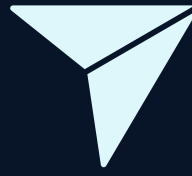
Thank You
Enjoy This?



Like



Comment



Share



Save



Dishant Kapoor

Contacts

dishantkapoor.com

info@dishantkapoor.com

<https://www.linkedin.com/in/dishantkapoor/>